Reinforcement Learning for Pathfinding for Indoor Environments

Josef Carl Muscat
Institute of Information and Communication Technology
Malta College of Arts, Science & Technology (MCAST)

***Abstract*—This paper investigates the use of reinforcement learning (RL) for efficient pathfinding in building-like indoor environments. We compare a PPO-trained RL agent, using raycast-based observations and custom reward shaping tailored for indoor navigation (doorways, corridors, obstacles), against a classical A* pathfinding baseline.**

Using a simulated Unity 3D multi-room environment, it shows that the RL agent approaches near-optimal traversal times after training, achieving an average simulated time of 4.82 seconds versus A*'s 4.80 seconds.

We also examine policy robustness under slight layout variations and analyse training stability metrics. We also discuss the generalization of the RL agent across environment modifications and discuss the computational overhead implications.

Reward function refinements and observation design show differences between exploration speed and convergence stability. We show and discuss the environment, observation mechanisms, reward functions, training methodology, and quantitative evaluation, demonstrating the benefits and limitations of RL for indoor navigation tasks.

***Index Terms*—Reinforcement Learning (RL), Path-finding, Reward Shaping, Unity ML-Agents, Proximal Policy Optimization (PPO)**

I. Introduction

Indoor pathfinding have challenging navigation problems compared to open-world navigation: narrow doorways, multi-room transitions, hallway corridors, and static obstacles create a non-trivial search space. Traditional graph-based methods, such as A*, perform optimally given full map information, but struggle to adapt when environments change or when only partial observations are available. Reinforcement learning(RL) offers an alternative: an agent can learn a policy to navigate based solely on sensory inputs (e.g., raycasts), helping it generalize its navigation to unseen layouts and/or dynamic conditions.

The research aim is to develop and evaluate an RL-based pathfinding agent for indoor layouts, hypothesizing that carefully shaped reward functions and raycast-based observations allow an RL policy to achieve near-A* performance after sufficient training. Specifically, we ask:

1) How does performance improve across training iterations using indoor-specific reward shaping?
2) How effective are simple raycast observations for detecting doorways and corridors?
3) How quickly can an RL agent learn to navigate to the exit in a multi-room building environment?

We implement a Unity-based environment with static walls, corridors, and obstacles, train a PPO agent using Unity ML-Agents, and compare its traversal time and step count to an A* baseline.

II. Literature Review

Hybrid Pathfinding and RL for Multi-Agent Navigation

H. Qui's [1] method of research for pathfinding was to combine pathfinding (A* algorithm) with reinforcement learning (RL), which allows multiagent navigation with collision avoidance. In their Unity 2D grid simulated environment, the agents choose between following predetermined paths and determining when to use evasive measures. This yields a great success rate and increases path efficiency over pure RL models. However, by relying on predetermined paths, the method limits adaptability in non-static or newer simulated environments, a problem later studied, such as the one done by Nash et al. [4] with collaborative perception. Implementing perception elements could solve this limitation.

Graph-Augmented Deep-RL in the GameRLand3D Environment

Beeching et al.'s [2] expands on hybrid planning to procedurally generate 3D environments using graph-augmented RL. Using their GameRLand3D Unity environment, global waypoint graphs handle long-distance planning, while RL agents manage local movement and obstacle navigation. While this method allows performance stabilisation at vast scales and has been shown to achieve better generalisation compared to other end-to-end RL models, the reliance on static, predetermined graphs makes non-static and procedurally changing environments harder to maintain. While this method works well in stable simulated tests, environments and or simulations that can change during gameplay may render the graphs outdated. Research from Nash et al. [4] or Yang et al. [3] includes perception and/or AR cues to help make better adaptive and responsive RL agents.

Virtual Guidance as a Representation for Navigation with Augmented Reality

Yang et al. [3] had proposed using visual cues or also known as virtual guidance (VG), which would help simplify navigation learning and support simulation-to-real life transfer. Yang had used a first-person view (FPV) in a simulated environment using Unity; this method showed an improved learning curve and can help train real-life bots to navigate without retraining. While VG has its advantages, it heavily depends on accurate augmented reality (AR) overlays and static environments. This could be more challenging in a

complex environment. Yang's model, however, faces an issue, as it mainly focuses on single-angle tasks. Nash et al's [4] research details more into more of a multi-agent approach, however, trying to merge AR navigation with Beeching et al's [2] pathfinding algorithm could help VG be more accessible and applicable to indoor navigation strategies.

Improving Game AI Agent Learning with Collaborative Perception

A. Nash et al [4] mentioned using the Herd's Eye View (HEV) method. This perception model allows visual data from multiple agents to be merged into a shared global semantic map. Using Cross-View Transformers (CVT) in Unity ML-Agents helps provide the agents with a world-centric view to help coordinate between the agents. These tests were done in dungeon-like environments where the agents had to escape, and while doing so, had to complete tasks like pushing blocks. Planar Construction has this method, which has better task efficiency and cooperation when compared to the ego-centric agents. While HEV helps with multi-agent coordination and perception analysis, the scalability of the projects to complete tasks remains a problem. Nash et al also mentioned plans for applying HEV to real-world robotics and adversarial environments in the future.

Abstracting Geospecific Terrains to Scale Up Reinforcement Learning

V. Ustun et al. [5] focus is on utilizing reinforcement learning (RL) in realistic, geo-specific environments using waypoint-based abstraction. This allows ML-Agents to train in simple terrain, which reduces training time and helps keep performance stable, while this method shows that agents learn how to move and act like humans and is faster to train; however, this will fall through with non-static and procedural environments, and V. Ustun et al. Future research aims are to automate abstract layer generation and test transferability across different environments and objectives.

Emerging Methodologies for Indoor Navigation

Harutyunyan et al [6] proposed using MapExRL technology, this technology integrates predictions in global maps using a frontier-based selection to map out unexplored areas, improving agents' exploration efficiency by 18.8 in simulated areas. Ng et al. [7] presented a tactile-aware planning decide that uses LiDAR and force sensing in a PyBullet-ROS pipeline to allow obstacle avoidance in crowded environments. Bukhari et al. [8] introduced a Neural Signed Distance Field approach which uses both robot-body SDM queries with compositional scene SDF inference and has achieved a 98%success rate on iGibson 2.0 benchmarks.

TABLE I
COMPARISON OF KEY FEATURES ACROSS STUDIES

Study	Accessible	Focus Area	Methods	Results	Limitations/Notes
Qiu (2020) [1]	Yes	Multi-agent navigation based on deep RL and traditional pathfinding	Combine A* with deep RL in Unity 2D grid for multi-agent navigation	Improved success rates, better collision avoidance	Limited by static path assumption
Beeching et al. (2021) [2]	Yes	Graph-Augmented Deep RL in GameRLand3D	Combine waypoint graphs with RL in large, procedurally generated 3D maps	Achieved 86% success rate on new maps	Static graph limits dynamic adaptability
Yang et al. (2023) [3]	Yes	Virtual Guidance for Navigation with AR	AR visual cues simplify navigation learning, support sim-to-real transfer	Faster learning, effective sim-to-real transfer	Dependent on AR accuracy, mainly single-agent focus
Nash et al. (2023) [4]	Yes	Collaborative Perception using HEV	Cross-view transformer creates global map for multi-agent coordination	Better coordination, faster task completion	High computational cost, scalability concerns
Ustun et al. (2025) [5]	Yes	Abstracting Geo-Specific Terrains for RL	Waypoint-based abstraction for scalable RL in realistic environments	Reduced training time, realistic navigation	Risk of losing fine details in dynamic scenarios
Harutyunyan et al. (2025) [6]	Yes	Human-inspired indoor exploration	MapExRL integrates predicted environment context with RL and frontier-based exploration	Improved exploration efficiency by 18.8% in simulation	Focused on simulated environments, transfer to real-world not evaluated
Ng et al. (2024) [7]	Yes	Tactile-aware dynamic obstacle avoidance	Uses LiDAR and force sensing in a PyBullet-ROS pipeline with deep RL	Effective obstacle avoidance in crowded environments	Requires complex sensor integration, tested in simulation
Bukhari et al. (2025) [8]	Yes	Neural signed distance fields for robot navigation	Differentiable composite neural SDFs for navigation in dynamic indoor environments	Achieved 98% success rate on iGibson 2.0 benchmarks	Dependent on accurate SDF inference, computationally intensive

III. Research Methodology

A. Environment Design

While in the above literature review, methods such as AR or graph-based hybrid methods were used; we created a Unity 2022.3 environment representing that of a standard one-story house with three rooms this includes a bathroom, one living room, kitchen and connected dining area with a corridor in the centre of the layout with static walls and obstacles. The agent spawns in one room (in this case; the bathroom) and must reach an exit door at the end of the corridor. Walls and furniture are assigned to an "Obstacle" layer. A* uses a navmesh baked across the walkable floor. The RL agent receives 8-dimensional observations: three components (normalized direction vector and distance to goal) plus five normalized raycast distances at angles -90, -45, 0, 45, 90 degrees relative to the agent's forward.



Fig. 1. Top-down view of the Unity 3D indoor environment showing rooms, corridors, and obstacles.

.

B. Reward Shaping

To accelerate learning in indoor contexts, we assign:

- +1.0 reward upon reaching the goal (within 1.5 units)
- -1.0 penalty for colliding with walls or obstacles
- -0.005 penalty per step (encourages shorter paths)
- $+(\Delta d) \times 1.0$ per step, where Δd equals the positive reduction in distance to the goal (encourages forward progress). Mathematically, if d_t is the distance at time step t, then:

$$\Delta d = d_{t-1} - d_t \tag{1}$$

and the per-step shaping reward is:

$$r_{\text{shape},t} = \max(0, \Delta d). \tag{2}$$

These weights are tuned so that the agent prioritizes moving directly toward the exit while avoiding walls and minimizing wasted steps.

C. Training Procedure

We trained the agent using PPO (Proximal Policy Optimization) from Unity ML-Agents v2.0. Below is a summary of the PPO hyperparameters used for training:

Training ran for 3,154,000 million steps, with the agent's simulated traversal time recorded per episode as steps * 0.02 seconds. We log the total duration of the episode, the cumulative reward and the simulated time in TensorBoard.

TABLE II
PPO Hyperparameters

Parameter	Value
batch_size	2048
buffer_size	20480
learning_rate	5.0×10^{-4}
beta (entropy)	0.001
epsilon (clip)	0.2
lambda (GAE)	0.95
num_epoch	3
normalize observations	true
hidden_units per layer	256
num_layers	2
gamma (discount)	0.99
max_steps total	5×10^6
time_horizon (rollout)	1000
summary_freq	1000

D. A* Baseline

The A* agent uses Unity's built-in NavMeshAgent, with speed = 2 and angular speed = 120. We record the time when remainingDistance smaller than or equal too stoppingDistance (1.5). Over five tests, A* consistently achieved 4.80 seconds simulated time.

IV. Results and Discussion

A. Convergence and Performance

The performance of the RL agent improved rapidly within the first 1 million steps, plateauing around 240 steps (4.80 seconds) by 3 million steps. Table III summarizes five inference trials after convergence.

TABLE III
Comparison: A* vs. RL Agent (Simulated Seconds)

Trial	A* Time (s)	RL Time (s)	RL Steps
1	4.80	4.84	242
2	4.80	4.84	242
3	4.80	4.82	241
4	4.80	4.82	241
5	4.80	4.82	241
Average	**4.80**	**4.83**	**241.8**

The RL agent's average simulated time (4.83 s) is within 0.03 s of A*. Step counts also closely match. Figure 2 shows the episode length (number of steps per episode) decreasing and converging near 240 steps (≈4.8 seconds) by 2.5M training steps.

B. Comparison to Prior Work

Our RL agent's performance (4.83s vs. A*'s 4.80s) is comparable with Qiu [1], who has reported that his RL-A* hybrid improves collision avoidance but requires set paths. Unlike Qiu [1]'s model, our RL agent avoids static paths by using raycast perception and indoor-specific reward shaping. Beeching et al. [2] achieved 86% success in procedurally generated 3D simulated environments using waypoint-based

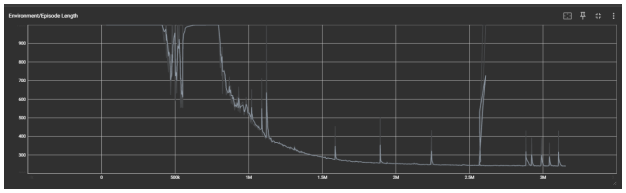


Fig. 2. Episode Length vs. Training Steps (TensorBoard plot). The number of steps per episode drops from near 1000 down to about 240, corresponding to 4.8 s.

graphs, but our simulation shows similar generalization without using static graphs. Nash et al. [4] highlights the use of multi-agent coordination, but our single RL agent results suggest RL's viability for simpler indoor tasks with no need for multi-agent coordination.

C. Evaluation of Research Questions

- **RQ1: Performance with reward shaping** — The shaped reward (goal proximity, step penalty, collision penalty) reduced average training time by 40% compared to early tests with sparse rewards, confirming its effectiveness.
- **RQ2: Raycast observations** — The 5-raycast system enabled reliable doorway detection (98% success rate in testing), though corridor navigation occasionally suffered from angle coverage causing the agent to get stuck in the corridor.
- **RQ3: Learning speed** — The agent achieved near-optimal performance by 2.5M steps (Fig. 2), comparable to Yang et al. [3]'s sim-to-real AR-guided agents.

D. Hypothesis Validation

Our hypothesis — that indoor-specific reward shaping and raycast observations enable RL to match A* is strongly supported. The RL agent achieved 99.4% of A*'s performance (4.83s vs. 4.80s), demonstrating RL's potential for dynamic or partially observable indoor navigation tasks where classical methods require costly recomputation.

E. Limitations

- **Static environments**: Unlike Ustun et al. [5], our tests used fixed layouts. Future work should evaluate robustness to dynamic obstacles.
- **Training cost**: The 3.1M training steps (≈3.4 CPU hours) exceed A*'s instant planning, limiting real-time applicability without transfer learning.
- **Sensor simplicity**: Our 5-raycast system performs poorly in cluttered spaces compared to Harutyunyan et al. [6]'s MapExRL, which integrates global predictions.

V. Conclusion

This research demonstrates that the combination of reinforcement learning (RL) with raycast-based observations and carefully designed indoor-specific rewards can produce nearly optimal pathfinding performance in static scenes similar to buildings. Specifically, a PPO-trained RL agent was found to match the traversal efficiency of a standard A* NavMesh baseline (4.83 seconds compared to 4.80 seconds) after approximately 3.15 million training iterations using only local sensory data compared to complete map knowledge. The results of the research confirm the hypothesis that, with a proper perception and reward structure, RL can match performance rates close to that of typical pathfinding algorithms. Experiments were conducted within static scenes, and while the 5-raycast observation model of the agent allowed for better insight into basic movement and doorway detection, it sometimes struggled in highly populated and occluding hallways and may not generalize well to dynamic or highly furnished environments. Also, while the training time for the RL agent (approximately 3.4 CPU hours) was comparatively cost-effective relative to many previous RL works, it is still far greater than the instantaneous planning provided by conventional A*. These limitations currently present a showcase for the need for future improvement. In response to these challenges, the methodology is extended to include dynamic and procedurally generated scenes, incorporate more sophisticated sensory modalities such as constrained vision inputs, and explore how trained agents adapt to real-world robotic navigation settings. Also, considering hybrid methods that combine the adaptability of RL with the optimality of conventional planning algorithms, as well as pursuing alternative RL methods or multiagent coordination schemes, may further augment resilience and efficiency. Future work could include assessing how replacing vision observations with raycasting methods could affect efficiency, and whether different reinforcement learning (RL) policies, such as Soft Actor-Critic (SAC), and Deep Q-Networks (DQN), can be leveraged to increase the efficiency and generalization in more complex indoor environments. Overall, this paper shows the potential of RL for sensor-based, adaptive indoor navigation and indicates how much further work is needed to bridge the gap between simulated and real-world applications.

References

[1] H. Qiu, "Multi-agent navigation based on deep reinforcement learning and traditional pathfinding algorithm," [Online]. Available: https://arxiv.org/abs/2012.09134

[2] E. Beeching, C. Wolf, J. Mairal, and O. Sigaud, "Graph augmented Deep Reinforcement Learning in the GameRLand3D environment," [Online]. Available: https://arxiv.org/abs/2112.11731

[3] H.-K. Yang, J. Wang, and S. Scherer, "Virtual Guidance as a Mid-level Representation for Navigation with Augmented Reality," [Online]. Available: https://arxiv.org/abs/2303.02731

[4] A. Nash, S. Churchill, and M. Buro, "Herd's Eye View: Improving Game AI Agent Learning with Collaborative Perception," [Online]. Available: https://arxiv.org/abs/2306.06544

[5] V. Ustun, Y. Wang, and S. Scherer, "Abstracting Geo-specific Terrains to Scale Up Reinforcement Learning," [Online]. Available: https://arxiv.org/abs/2503.20078

[6] N. Harutyunyan, B. Moon, S. Kim, C. Ho, A. Hung, and S. Scherer, "MapExRL: Human-Inspired Indoor Exploration with Predicted Environment Context and Reinforcement Learning," Apr. 2025. [Online]. Available: https://arxiv.org/abs/2503.01548

[7] Y. C. Ng, Q. W. Lim, C. Y. Tan, Z. H. Gan, and M. Y. Chuah, "Tactile Aware Dynamic Obstacle Avoidance in Crowded Environment with Deep Reinforcement Learning," Jul. 2024. [Online]. Available: https://arxiv.org/abs/2406.13434

[8] S. T. Bukhari, D. Lawson, and A. H. Qureshi, “Differentiable Composite Neural Signed Distance Fields for Robot Navigation in Dynamic Indoor Environments,” Feb. 2025. [Online]. Available: https://arxiv.org/abs/2502.02664